

# Control Flow Integrity

## Project Summary

**Institution:** University of Texas at Dallas - Team b00n

**Team Members:** Ian Brown [ixb130430@utdallas.edu](mailto:ixb130430@utdallas.edu)

Anastasia Hudman [sxh122630@utdallas.edu](mailto:sxh122630@utdallas.edu)

Joel Seida [jps140430@utdallas.edu](mailto:jps140430@utdallas.edu)

Jeremiah Shipman [jxs116830@utdallas.edu](mailto:jxs116830@utdallas.edu)

Corrin Thompson [cnt130030@utdallas.edu](mailto:cnt130030@utdallas.edu)

**Document:** Progress Report

**Keywords:** Control Flow Integrity (CFI), Buffer Overflow, Memory Corruption  
Exploitation, Shadow Stack, Landing Point Instruction, Return Oriented Programming  
(ROP)

**Project Description:** NSA has developed techniques—landing point instructions and shadow stack—to enforce control flow integrity. This project will demonstrate the strength of these mitigations through attempted exploitation.

**Deliverables:**

- Proposal & Literature Review
- Progress Report & Presentation
- Final Report & Presentation
- Documentation & Code

# Control Flow Integrity

## Executive Summary

**Institution:** University of Texas at Dallas - Team b00n

**Team Members:** Ian Brown [ixb130430@utdallas.edu](mailto:ixb130430@utdallas.edu)

Anastasia Hudman [sxh122630@utdallas.edu](mailto:sxh122630@utdallas.edu)

Joel Seida [jps140430@utdallas.edu](mailto:jps140430@utdallas.edu)

Jeremiah Shipman [jxs116830@utdallas.edu](mailto:jxs116830@utdallas.edu)

Corrin Thompson [cnt130030@utdallas.edu](mailto:cnt130030@utdallas.edu)

**Keywords:** Control Flow Integrity (CFI), Buffer Overflow, Memory Corruption  
Exploitation, Shadow Stack, Landing Point Instruction, Return Oriented Programming  
(ROP)

### Summary:

Control flow integrity refers to the protection of the sequence of instructions in a program. Currently, existing control flow integrity mitigations only make it harder for the attacker to gain a foothold, rather than preventing access entirely. In order to address this issue, the NSA has created techniques that can be overlaid in a binary to enforce control flow integrity. These techniques include the landing point instructions and the shadow stack. The goal of this project involves the demonstration of the strength or weakness of these techniques through attempted exploitation of a suspected vulnerability. Analysis tools will be used to determine if the mitigations can be bypassed using the vulnerability. If successful, a proof of concept will be used to demonstrate the complete attack.

## Table of Contents

|                            |    |
|----------------------------|----|
| Introduction.....          | 4  |
| Problem Statement.....     | 4  |
| Purpose Statement.....     | 4  |
| Motivation.....            | 4  |
| Impact.....                | 4  |
| Literature Review.....     | 5  |
| Progress.....              | 7  |
| Plan Overview.....         | 7  |
| Detailed Plan.....         | 7  |
| Work completed.....        | 7  |
| Work in progress.....      | 8  |
| Work to be completed.....  | 8  |
| Deliverables.....          | 9  |
| Schedule.....              | 9  |
| Results to Date.....       | 9  |
| Issues.....                | 10 |
| References.....            | 10 |
| Team.....                  | 11 |
| Biographical sketches..... | 11 |
| Ian Brown.....             | 11 |
| Anastasia Hudman.....      | 11 |
| Joel Seida.....            | 12 |
| Jeremiah Shipman.....      | 12 |
| Corrin Thompson.....       | 12 |
| Tasking.....               | 13 |

## Introduction

### Problem Statement

Malicious software can take advantage of vulnerabilities in a program by redirecting its flow of execution (the control flow). Control flow integrity (CFI) is designed to prevent these types of attacks. NSA developed two CFI techniques: landing point instructions and shadow stack. The objective is to test one of the landing point instruction technique in order to demonstrate its strengths and weaknesses.

### Purpose Statement

This project aims to test the resilience of the landing point instruction safety technique by chaining gadgets (small snippets of code) in order to manipulate a return address in a meaningful way. If a return address can be manipulated, then the CFI technique can be shown to be ineffective.

### Motivation

Control flow integrity is vital to the prevention of the spread of malware. There are many different implementations that attempt to prevent an attacker from hijacking program flow and taking it down an unintended path. However, in the current situation, most control flow integrity mechanisms do not address the issue of control flow directly. Instead, these mitigations only serve to make it harder for an attacker to maintain control, but do not entirely prevent the attacker from gaining control in the first place. Other solutions must be considered in order to properly maintain control flow integrity.

The two techniques developed by NSA (landing point instructions and shadow stack) are a crucial step in preventing the next level of malware attacks. However, they cannot be implemented without thorough testing and evaluation. This project will attempt to uncover potential flaws in the design of these mechanisms in order to

help demonstrate their weakness, or perhaps their strength. If these techniques are proven worthy, they can become a boon to our society.

## Impact

The research conducted in this semester will provide further evidence to prove or disprove the security that the shadow stack and landing point instructions provide. While the scope of this project has been limited to the study of gadget chaining, the tool might be able to be expanded for further use by other teams.

Demonstrating the effectiveness of these CFI methodologies could facilitate the development of new CFI methodologies and make them commonplace in future software products. Eventually, if these CFI techniques lead to a system that is completely secure against ROP/COP/JOP attacks, this research will have been a boon to society.

## Literature Review

### Relevance

Control flow Integrity (CFI) is the series of security techniques to prevent arbitrary code execution by altering the flow of the program. Attackers may be able to alter the control flow of a program using variations of a buffer overflow attack, where a return address is overwritten, and points to malicious code to be executed<sup>[1]</sup>.

Despite buffer overflows being fairly basic and well-known vulnerabilities, they are prevalent in modern software. Between 1988 and 2012, buffer overflows accounted for the largest portion of reported vulnerabilities (about 14%). Of vulnerabilities with a "high" severity rating, buffer overflows make up 23%, and of critical vulnerabilities they consist of 35%<sup>[2]</sup>. A well-known example of this attack is the Twilight Hack used to homebrew a Wii console (developed in 2008). This attack overloads the value of a character's name, which when loaded into memory, spills into adjacent buffers, allowing arbitrary programs to be executed<sup>[4]</sup>.

Developing CFI attacks that are infeasible to break is crucial to protecting company and organizational software as new attack methods are developed.

### **Control Flow Integrity Methods**

In June 2016, Intel released a document detailing a new Control-flow Enforcement Technology (CET) to protect against return-oriented programming (ROP) attacks and call/jmp-oriented programming (COP/JOP) <sup>[3]</sup>. These types of attack techniques are typically able to divert control flow from its original target address to a different target by changing the data stack or register. The techniques also have a module of code that possesses execution privileges and code containing control transfer instructions that rely on data either in the return stack or in a target address register.

Intel's CET contains two mechanisms, shadow stack and indirect branch tracking, to protect against ROP and COP/JOP style attacks. The Shadow Stack technique is a return address protection used to protect against ROP attacks. The indirect branch tracking technique provides free branch defense to protect against JOP attacks.

### **NSA Proposed Methods**

NSA proposes a protected shadow stack in combination with using landing point instructions. The protected shadow stack is in a secure area of memory, separate from the regular stack. When a return address is stored in the regular stack, it is also stored in the shadow stack. Upon returning, the return value is compared to the value in the shadow stack.

Landing point instructions are newly proposed instructions for the x86 architecture. Whenever a branch (jump, call, return) statement occurs, the branch's destination

must be a corresponding landing point instruction. With these two features, NSA hopes to eliminate ROP, JOP, and COP attack techniques<sup>[5]</sup>.

### **Previous Group Work**

The group from CMU that worked on the project in Fall 2016 focused on a method of ranking gadgets by intensity of side-effects. Over the course of the semester, they produced a tool capable of accomplishing this goal. Due to the short time frame of the project, they were unable to add all intended features of the tool.

The group from DSU that worked on CFI in Spring 2016 focused on attempting to prove Turing-completeness for gadgets in supplied binaries. While their findings were inconclusive, they did show that Turing-complete gadget sets would be hard to find.

### **Team Approach and Justification**

The team will create a tool capable of chaining together gadgets. The tool will attempt to circumvent NSA's control flow integrity techniques (landing point instructions and the shadow stack) in order to gain control of the application using this process.

To create this tool, the team will first analyze and attempt to manually determine potentially feasible gadget chains. The tool will then be created in order to automatically execute these pre-written chains. If time permits, the tool may be expanded to accommodate the usage of more gadgets. Otherwise the source code will be provided for future teams to expand on this work.

## Progress

### Plan Overview

The overarching goal of this project is to prove the feasibility (or infeasibility) of breaking the landing point instruction CFI method designed by NSA. Accomplishing this goal requires sorting through and testing a wide array of pre-defined gadgets. To ease the process of testing these gadgets, and to enhance the productivity of future groups, the team will devote their semester to accomplishing a smaller portion of this goal: chaining together gadgets to attempt to break the CFI methods.

To ensure completion this sub-task, a gadget-chaining tool will be developed. Success will be determined by how well we meet, or exceed, the deliverables defined in this proposal. Additionally the project will be deemed successful if the tool produced is useful to future teams in their research into this problem.

### Detailed Plan

#### Work completed

**Task:** Create tool

**Description:** Develop a tool that will split gadget files into individually testable components

**Headline:** The tool is complete. It successfully stripped supplied files, dividing them into individual files, sorting them by ease of testing. Results were as expected.

**Detailed Description:** The tool was written as a series of bash scripts that split and trim the gadgets. All files are hosted on GitHub. The tool was run over the course of a weekend. The stripped gadgets were divided into two categories: easy and splitStrip. After that, we executed each gadget in a C program that tests whether or not the EAX register was incremented or decremented by 1.

**Outcomes:** The results of this task were individual gadget files to be used later on in the project. The “easy” folder contains gadgets that have no jump operations, and will be simple to manually assess and test against the provided Linux binaries.

### Work in progress

**Task:** Manually assessing feasibility of gadgets

**Description:** Team will manually analyze a series of gadgets and manually analyze them for feasibility of breaking landing point instruction technique

**Headline:** Task has not been completed at this time. Result will be a series of simple gadgets to test against provided Linux binary.

**Detailed Description:** This task involves printing physical copies of the gadgets. Each member of the team will analyze these gadgets and assess their potential success in breaking the landing point instruction technique. These gadgets identified will be tested later.

**Outcomes:** The result will be list of easily testable gadgets.

### Work to be completed

**Task:** Chain gadgets to break landing point instructions

**Description:** Test gadgets in succession to see if vulnerabilities in the Linux binary can be located.

**Headline:** Task not yet initiated. End goal is to see whether subset of gadgets can find vulnerabilities in control flow integrity techniques.

**Detailed Description:** A tool will be developed to automatically test these gadgets against the provided binary.

**Outcomes:** Results include a report documenting what attacks were feasible.

### Deliverables

- Proposal & Literature Review
- Progress Report & Presentation
- Final Report & Presentation

- Documentation & Code

## Schedule

Below is a weekly schedule listing what goals will be accomplished by the Friday of that week.

Feb 17: Submit Proposal

Feb 24: Review project documentation, study how different gadgets work

Mar 3: Draft tool design, define set of gadgets to be used

Mar 10: Finalize tool design and assign roles to members, implementation of design

Mar 17: Implementation of design

Mar 24: Prepare presentation and progress report

Mar 31: Submit progress report and presentation

Apr 7: Analyze individual gadgets

Apr 14: Test gadgets against provided Linux binary file

Apr 21: Additional testing and final report draft

May 5: Submit final report and presentation

## Results to Date

As of the creation of this document (submitted March 31, 2017), team b00n has created a tool capable of splitting and categorizing individual gadgets from the provided files. Gadgets containing jump, call, and return statements are deemed "hard", while those that do not contain such statements are deemed "easy." Additionally, the tool strips out all jump, call, and return statements. The team assumes that the gadget has been implemented correctly, and thus strips out these statements to make testing more efficient. Our other tool can take a gadget and run it inside of a C program (so that we can execute the assembly). With this tool, we

can analyze how key registers change after the gadget has been executed. Currently, we are hoping to find a gadget that can increment or decrement the EAX register by 1 without changing the EAX register anywhere else or otherwise crashing.

## Issues

Issues encountered:

- Difficulty in breaking theoretical implementations of defenses
- Several hundred thousand gadgets to analyze and test within limited time constraints
- Landing Point Instruction technique has already been thoroughly tested, making generation of original tests challenging

Issues regarding differences in team skillsets did arise as predicted. Transfer of knowledge has not been as efficient as desired, due to conflicting exam schedules and multiple external obligations by individual team members. In the following month, individual team member schedules become less hectic, and progress lost due to differing skillsets can be remedied within the allotted time.

## References

- [1] M. Abadi, M. Budiu, Ú. Erlingsson and J. Ligatti, "Control-flow integrity principles, implementations, and applications", ACM Transactions on Information and System Security, vol. 13, no. 1, pp. 1-40, 2009.
- [2] Y. Younan, "25 years of vulnerabilities: 1988 - 2012," SourceFire, 2013.
- [3] Intel, "Control-flow Enforcement Technology Preview", intel.com, 2016. [Online]. Available:  
<https://software.intel.com/sites/default/files/managed/4d/2a/control-flow-enforcement-technology-preview.pdf>. [Accessed: Feb. 9, 2017].

[4] "Twilight hack," in wiibrew.org, 2014. [Online]. Available:

[http://www.wiibrew.org/wiki/Twilight\\_Hack](http://www.wiibrew.org/wiki/Twilight_Hack). [Accessed: Feb. 9, 2017].

[5] NSA, "Hardware Control Flow Integrity for an IT Ecosystem," in github.com,

GitHub, 2015. [Online]. Available: <https://github.com/iadgov/Control-Flow-Integrity/blob/master/paper/Hardware%20Control%20Flow%20Integrity%20for%20an%20IT%20Ecosystem.pdf>

0Ecosystem.pdf. Accessed: [Accessed: Feb. 9, 2017].

## Team

### Biographical sketches

#### Ian Brown

Ian Brown holds a Bachelor's degree in Software Engineering with a specialization in computer networks and is currently pursuing a Master's degree in Computer Science with a specialization in information assurance. He is a recipient of the NSF's Scholarship for Service. Ian has a wide variety of security experience including binary exploitation, web/application/network penetration testing, and secure programming. He has completed coursework in computer networks, operating systems, MIPS assembly language, operating systems, information security, and network security. Beyond the classroom, Ian has also held the secretary and president positions within the Computer Security Group at UT Dallas and is studying for the OSCP certification.

#### Anastasia Hudman

Anastasia Hudman holds a Bachelor's degree in Software Engineering and is currently pursuing a Master's degree in Computer Science with a specialization in information assurance. She is a CyberCorps: Scholarship for Service recipient and has completed an internship for the Department of Defense as a Blue Team Forensic Analyst. She has experience in memory analysis, virtual machines, software development, and project management. She has taken courses in data and

applications security, information security, operating systems, computer networks, and database design. Her knowledge of programming languages includes Java, C++, and C. She held officer roles in the Computer Security Group at UT Dallas and helped create challenges for a Capture the Flag competition. She also volunteered at the Women in Cybersecurity Conference in Spring 2016.

### Joel Seida

Joel Seida holds a Bachelor's degree in Computer Science and is currently pursuing a Master's degree in Computer Science with a specialization in Information Assurance. He is a recipient of the NSF's Scholarship for Service and has completed an internship for Sandia National Labs as a part of the Center for Cyber Defenders. He has also taken coursework in data and applications security, information security, operating systems, computer networks, distributed computing, network security, database design, algorithm design, and is currently taking a class on computation theory.

### Jeremiah Shipman

Jeremiah Shipman holds a Bachelor's degree in Computer Science and is currently pursuing a Master's degree in Computer Science with a specialization in Information Assurance. He is a recipient of the NSF's Scholarship for Service and has completed an internship for the Los Alamos National Laboratory. He has experience solving cyber-security problems in CTF (capture-the-flag) competitions (including ones involving buffer overflows). He has related course experience in operating systems, MIPS assembly language, information security, and network security. He is also currently preparing for the OSCP online course and certification.

### Corrin Thompson

Corrin Thompson is a senior-level software engineering major, with a specialization in information assurance. She is currently a fast track student and working on a

master's degree in computer science while finishing undergraduate studies. Since freshman year, Corrin has been involved with a local student organization on campus: the Computer Security Group (CSG). Her work in the group has given her experience in topical areas of computer security, as well as leadership and communication. Her current specializations are digital forensics and web security. In addition she has also worked on the Security & Architecture team with Liberty Mutual Insurance, performing vulnerability analysis on pre-production code and developing tools to aid in incident response.

## Tasking

Member roles will vary based on the phases of the project as the group learns each members' strengths and weaknesses. The three software engineering majors, Ian Brown, Anastasia Hudman, and Corrin Thompson, will handle most of the documentation, requirements analysis, and testing as the project progresses. The two computer science majors, Joel Seida and Jeremiah Shipman, will lead in design and implementation of code. Every member will contribute in devising an attack to attempt to bypass the mitigations.